Trabalho Final: GiroBlu

[Integrante 1], [Integrante 2], [Integrante 3]

Programação Orientada a Objetos 2026/2

FURB
UNIVERSIDADE DE BLUMENAU

Antes de começar (apague este slide)

Tempo máximo: 10 minutos

cerca de 8 min de apresentação e 2 min de perguntas

Não mude a ordem nem os títulos

troque tudo o que está entre [colchetes] pelo conteúdo da equipe

Todos apresentam

cada integrante apresenta pelo menos um slide; preencha o campo "Apresenta" no rodapé

Código legível

trechos curtos, fonte de no mínimo 14 pt; nada de colar a classe inteira

Slide	Tempo
Capa	0:15
Diagrama de classes	1:30
Decisões de modelagem	1:00
Onde usamos cada conceito	1:30
Polimorfismo em ação	1:00
Plano de testes	0:45
Testes implementados	0:45
Execução dos testes	0:30
Como foi aplicar os conceitos	0:45
Total	8:00

Diagrama de classes

[Cole aqui o diagrama de classes UML da camada de negócio]

Se o diagrama completo ficar pequeno demais para ler, mostre aqui uma visão geral e abra o arquivo original durante a apresentação.

Apresenta: *[nome do integrante]*

Decisões de modelagem

1

Como representamos os tipos de veículo

[Explique a solução em uma ou duas frases e por que a equipe escolheu esse caminho]

2

Como representamos os planos dos usuários

[Explique a solução e como ela trata a regra do Universitário com patinete]

3

Uma alternativa que consideramos e descartamos

[O que era, e por que não seguimos com ela]

Apresenta: *[nome do integrante]*

Onde usamos cada conceito

Conceito	Onde (classes e métodos)	Por que ali
Encapsulamento	*[Classe X, método y()]*	*[O problema que isso resolveu]*
Herança	*[Classe X, método y()]*	*[O problema que isso resolveu]*
Classe abstrata	*[Classe X, método y()]*	*[O problema que isso resolveu]*
Método abstrato	*[Classe X, método y()]*	*[O problema que isso resolveu]*
Polimorfismo	*[Classe X, método y()]*	*[O problema que isso resolveu]*
Interface	*[Classe X, método y()]*	*[O problema que isso resolveu]*

Apresenta: *[nome do integrante]*

Polimorfismo em ação

```
// [Cole aqui um trecho curto,
//  de até 12 linhas, em que a
//  mesma chamada se comporta de
//  forma diferente conforme o
//  objeto que a recebe]
```

O que acontece aqui

[Qual método é chamado, quais classes o sobrescrevem e o que muda no resultado]

Apresenta: *[nome do integrante]*

Plano de testes (amostra)

ID	Classe e método	Cenário	Entrada	Resultado esperado
[CT01]	*[Classe.metodo()]*	*[O que é verificado]*	*[Dados usados]*	*[Valor ou exceção]*
[CT02]	*[Classe.metodo()]*	*[O que é verificado]*	*[Dados usados]*	*[Valor ou exceção]*
[CT03]	*[Classe.metodo()]*	*[O que é verificado]*	*[Dados usados]*	*[Valor ou exceção]*
[CT04]	*[Classe.metodo()]*	*[O que é verificado]*	*[Dados usados]*	*[Valor ou exceção]*

Plano completo: *[N] casos de teste cobrindo [N] métodos da camada de negócio*

Apresenta: *[nome do integrante]*

Testes implementados

Implementa o caso: *[CT0X]*

```
@Test
void [nomeDoTeste]() {
    // [Cole aqui um ou dois métodos de teste JUnit
    //  que implementam casos do plano mostrado
    //  no slide anterior]
}
```

Apresenta: *[nome do integrante]*

Execução dos testes

[Print da execução dos testes na IDE]

Com a barra de resultado e a lista de testes visíveis

[N] testes executados

[N] passaram

[N] falharam

Algum falhou? *[Explique o motivo]*

Apresenta: *[nome do integrante]*

Como foi aplicar os conceitos

1

O mais difícil

[Qual conceito ou parte do sistema deu mais trabalho, e por quê]

2

O que aprendemos

[O que ficou claro só na hora de implementar]

3

O que faríamos diferente

[Se começassem de novo, o que mudariam no projeto]

Apresenta: *[nome do integrante]*